VpgGraph.tsx

frontend/components | TypeScript / React

```tsx
/**
 * VpgGraph.tsx
 *
 * Real-time Vascular Pressure Graph — the visual heart of BioSync.
 *
 * Renders the rPPG-derived pulse waveform using the HTML5 Canvas API.
 * Color, amplitude, and noise level all respond to the current stress state,
 * creating a live visual feedback loop that mirrors the user's cognitive load.
 *
 * Design decisions:
 *  - Canvas API (not SVG) for 60fps rendering without DOM overhead
 *  - requestAnimationFrame loop for browser-synchronized repaints
 *  - Separate noise model for CALM vs CRITICAL states (visual differentiation)
 *  - Signal confidence overlay communicates data quality honestly
 */

import React, { useEffect, useRef, useCallback } from 'react';

// ──── Types ──────────────────────────────────────────────────────────────

interface VpgGraphProps {
  /** Current cognitive stress score (0–100) */
  stressLevel: number;
  /** Whether Wi-Fi is off — triggers EDGE_ACTIVE label */
  isOffline: boolean;
  /** Current OS state — drives waveform color */
  state: 'CALM' | 'LOAD' | 'CRITICAL';
  /** Signal confidence percentage (0–100) */
  signalConfidence: number;
}

// ──── Constants ──────────────────────────────────────────────────────────

/** Waveform colors mapped to cognitive state */
const STATE_COLORS = {
  CALM: '#22d3ee',     // Cyan — hemmed, stable
  LOAD: '#facc15',     // Yellow — elevated attention
  CRITICAL: '#ef4444', // Red — high stress
} as const;

/** How many data points to render (width of the rolling window) */
const BUFFER_SIZE = 120;

/** Canvas dimensions */
const CANVAS_WIDTH = 600;
const CANVAS_HEIGHT = 160;

// ──── Component ──────────────────────────────────────────────────────────

export default function VpgGraph({
  stressLevel,
  isOffline,
  state,
  signalConfidence,
}: VpgGraphProps) {
  const canvasRef = useRef<HTMLCanvasElement>(null);
  const dataBuffer = useRef<number[]>([]);
  const animFrameId = useRef<number>(0);

  /**
   * Generates the next waveform sample.
   *
   * Combines a physiologically plausible sine wave (simulating the cardiac
   * cycle) with stress-proportional noise. At CALM state, the wave is smooth
   * and regular. At CRITICAL, it becomes erratic and high-amplitude —
   * visually communicating sympathetic nervous system activation.
   */
```

```
const generateSample = useCallback((): number => {
const t = Date.now();

// Base cardiac wave: ~1 Hz (60 BPM), 20px amplitude
const baseWave = Math.sin(t / 160) * 20;

// Stress-proportional noise: scales from ±2px (CALM) to ±12px (CRITICAL)
const noiseAmplitude = 2 + (stressLevel / 100) * 10;
const noise = (Math.random() - 0.5) * noiseAmplitude;

// Secondary harmonic: adds realistic complexity to the pulse shape
const harmonic = Math.sin(t / 80) * (stressLevel / 100) * 5;

return baseWave + noise + harmonic;
}, [stressLevel]);

/**
* Main render loop.
*
* Runs at the browser's animation frame rate (typically 60fps).
* Each frame: shift buffer, add new sample, redraw entire waveform.
*/
const render = useCallback(() => {
const canvas = canvasRef.current;
if (!canvas) return;
const ctx = canvas.getContext('2d');
if (!ctx) return;

// 1. Maintain rolling buffer
if (dataBuffer.current.length >= BUFFER_SIZE) {
dataBuffer.current.shift();
}
dataBuffer.current.push(generateSample());

// 2. Clear canvas with semi-transparent fill (creates motion trail effect)
ctx.fillStyle = 'rgba(2, 6, 23, 0.15)';
ctx.fillRect(0, 0, CANVAS_WIDTH, CANVAS_HEIGHT);

// 3. Draw waveform
const waveColor = STATE_COLORS[state];
ctx.beginPath();
ctx.strokeStyle = waveColor;
ctx.lineWidth = state === 'CRITICAL' ? 2.5 : 2;
ctx.lineJoin = 'round';
ctx.lineCap = 'round';

// Optional glow effect for CRITICAL state
if (state === 'CRITICAL') {
ctx.shadowColor = waveColor;
ctx.shadowBlur = 8;
} else {
ctx.shadowBlur = 0;
}

dataBuffer.current.forEach((point, index) => {
const x = (index / (BUFFER_SIZE - 1)) * CANVAS_WIDTH;
const y = CANVAS_HEIGHT / 2 + point;

if (index === 0) ctx.moveTo(x, y);
else ctx.lineTo(x, y);
});

ctx.stroke();
ctx.shadowBlur = 0; // Reset shadow for labels

// 4. Draw telemetry overlays
ctx.font = '10px monospace';
ctx.fillStyle = isOffline ? '#4ade80' : '#475569'; // Green if offline (edge active)
ctx.fillText(isOffline ? 'EDGE_ACTIVE ●' : 'CLOUD_SYNC', 10, 18);

const confidenceColor = signalConfidence > 70 ? '#22d3ee' : '#f59e0b';
ctx.fillStyle = confidenceColor;
ctx.fillText(`SIGNAL_CONFIDENCE: ${signalConfidence}%`, 10, 34);

ctx.fillStyle = '#334155';
ctx.fillText(`rPPG_CHANNEL | GREEN_EXTRACT | KALMAN_FILTERED`, 10, CANVAS_HEIGHT - 10);

// 5. Schedule next frame
```

```
animFrameId.current = requestAnimationFrame(render);
}, [generateSample, isOffline, state, signalConfidence]);

// Start/stop the animation loop when the component mounts/unmounts
useEffect(() => {
animFrameId.current = requestAnimationFrame(render);
return () => cancelAnimationFrame(animFrameId.current);
}, [render]);

return (
<canvas
ref={canvasRef}
width={CANVAS_WIDTH}
height={CANVAS_HEIGHT}
className="w-full rounded-lg bg-slate-950"
aria-label="Real-time vascular pressure graph"
/>
);
}
```